

Recall: An algorithm is called backward stable (on the input x) if the computed output $\tilde{y}(x)$ (computed in machine arithmetic) can be seen as the exact output $y(\hat{x})$, for some $\hat{x}$ s.t. $\frac{\|\hat{x} - x\|}{\|x\|} = O(u)$.

$C = AB$ matrix-matrix product $\leadsto \tilde{C}$

$$c_{ij} = A(i, :) B(:, j)$$

$$\leadsto |\tilde{c}_{ij} - c_{ij}| \leq O(u) \cdot [|a_{i1}| |a_{i2}| \dots |a_{in}|] \begin{bmatrix} |b_{1j}| \\ |b_{2j}| \\ \vdots \\ |b_{nj}| \end{bmatrix}$$

$$\|\tilde{C} - C\|_F = \sqrt{\sum_{i,j} |\tilde{c}_{ij} - c_{ij}|^2} \leq \dots \leq O(u) \cdot \|A\|_F \cdot \|B\|_F$$

$\leadsto$ the relative error can be quite large... $\ddot{\smile}$

If A is square and invertible:

$$\tilde{C} = AB + F \quad \|F\| = \|\tilde{C} - C\| = O(u) \cdot \|A\| \cdot \|B\|$$

$$\hookrightarrow = A(\underbrace{B + A^{-1}F}_{\hat{B}})$$

$$\leadsto \tilde{C} = A\hat{B}$$

$$\frac{\|\hat{B} - B\|}{\|B\|} = \frac{\|A^{-1}F\|}{\|B\|} \leq \frac{\|A^{-1}\| \cdot \|F\|}{\|B\|} \leq \frac{\|A^{-1}\| \|A\| \cdot \|B\| \cdot O(u)}{\|B\|} = \kappa(A) \cdot O(u)$$

Theorem: if $Q \in \mathbb{R}^{n \times n}$ is orthogonal, and B is any matrix of compatible size, then computing $C = QB$ is backward stable, that is, $\tilde{C} = Q\hat{B}$ for some $\hat{B}$ s.t. $\frac{\|B - \hat{B}\|}{\|B\|} = O(u)$.

The same holds if $Q = I - 2uu^T$ for some unit vector u and $C = B - 2u \cdot (u^T B)$: the obtained $\tilde{C}$ satisfies

$$\tilde{C} = H\hat{B} \text{ for some } \hat{B} \text{ s.t. } \frac{\|B - \hat{B}\|}{\|B\|} = O(u).$$

THEOREM: [QR factorization is backward stable]

The computed $\tilde{R}$, $\tilde{u}_1, \tilde{u}_2, \dots, \tilde{u}_n$ satisfy

$$\hat{A} = Q(\tilde{u}_1) Q(\tilde{u}_2) \dots Q(\tilde{u}_n) \tilde{R} \text{ for a matrix } \hat{A}$$

such that $\frac{\|\hat{A} - A\|}{\|A\|} = O(u)$.
this is an orthogonal matrix upper triangular

$$Q(\tilde{u}_j) = \begin{bmatrix} I_{j-1} & 0 \\ 0 & I_{n-j+1} - 2\tilde{u}_j\tilde{u}_j^T \end{bmatrix}$$

$\tilde{u}_j \in \mathbb{R}^{n-j+1}$

Sketch of the proof:

$$R_0 = A \rightsquigarrow R_1 = \begin{bmatrix} * & \text{---} \\ 0 & \text{---} \\ 0 & \text{---} \\ 0 & \text{---} \end{bmatrix} \rightsquigarrow R_2 = \begin{bmatrix} * & * & \text{---} \\ 0 & * & \text{---} \\ 0 & 0 & \text{---} \\ 0 & 0 & \text{---} \end{bmatrix} \rightsquigarrow R_n = R = \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix}$$

$$R_0 = A$$

$$R_i = Q(u_i) \cdot R_{i-1}$$

In machine arithmetic: $R_0 = A$

$$\text{step 1: } \tilde{R}_1 = Q(\tilde{u}_1)(A + \Delta_0)$$

$$\begin{aligned} \text{step 2: } \tilde{R}_2 &= Q(\tilde{u}_2)(\tilde{R}_1 + \Delta_1) = Q(\tilde{u}_2)(Q(\tilde{u}_1)A + Q(\tilde{u}_1)\Delta_0 + \Delta_1) \\ &= Q(\tilde{u}_2)Q(\tilde{u}_1)(A + \Delta_0 + Q(\tilde{u}_1)\Delta_1) \end{aligned}$$

$$\text{step 3: } \tilde{R}_3 = Q(\tilde{u}_3)(\tilde{R}_2 + \Delta_2) = Q(\tilde{u}_3)Q(\tilde{u}_2)Q(\tilde{u}_1)(A + \Delta_0 + Q(\tilde{u}_1)\Delta_1 + Q(\tilde{u}_1)Q(\tilde{u}_2)\Delta_2)$$

$$\|\Delta_0\| = \|A\| \cdot O(u)$$

$$\|\Delta_1\| = \|\tilde{R}_1\| \cdot O(u) = \|R_1\| \cdot O(u) + O(u^2) = \|A\| \cdot O(u)$$

because $R_1 = Q(u_1)A$
and multiplying by
orth. matrices does not
change the norm

... all the Δ_j errors are of the order of $\|A\| \cdot O(u)$

Backward stability at each step, orthogonal transformation
 $\Rightarrow$ backward stability!

$$\tilde{R}_n = \underbrace{Q(\tilde{u}_n)Q(\tilde{u}_{n-1})\dots Q(\tilde{u}_1)}_{\text{orthogonal (exactly)}} \underbrace{(A + \Delta_0 + Q(\tilde{u}_1)\Delta_1 + \dots + \tilde{u}_n\Delta_n)}_{O(u) \cdot \|A\|}$$

let this matrix be $\hat{A}$
In this way, $\frac{\|\hat{A} - A\|}{\|A\|} = O(u)$

$$\text{Moreover, } \hat{A} = Q(\tilde{u}_1)Q(\tilde{u}_2)\dots Q(\tilde{u}_n)\tilde{R}_n$$

[This is saying that the Householder QR algorithm, in machine arithmetic, is computing the exact QR factorization of a matrix $\hat{A}$ which is very close (in a relative sense) to the matrix A we were interested in.] □

To solve $\min_x \frac{1}{2} \|Ax - b\|^2$ via QR factorization:

$$\bullet A = Q_0 R_0$$

(thin QR fact.)

backward stable ;)

$$\bullet c = Q_0^T b$$

backward stable ;)

$$\bullet x = R_0^{-1} c \rightsquigarrow \text{upper triangular lin. system}$$

also backward stable!

THEOREM [Back substitution for upper triangular linear systems is backward stable]

$x = f(T, b)$ solution of the lin. system $Tx = b$

$$T = \begin{bmatrix} t_{11} & t_{12} & \dots & t_{1n} \\ 0 & t_{22} & \dots & t_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & t_{nn} \end{bmatrix}$$

Then $\tilde{x} = f(\hat{T}, b)$ for some $\hat{T}$ s.t. $\frac{\|\hat{T} - T\|}{\|T\|} = O(u)$

Proof: $T = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ 0 & t_{22} & t_{23} \\ 0 & 0 & t_{33} \end{bmatrix}$ $b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$ $x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$

$$x_3 = b_3 / t_{33}$$

$$\tilde{x}_3 = b_3 \oslash t_{33} = \frac{b_3}{t_{33}} (1 + \delta_1) \quad |\delta_1| < u$$

$$x_2 = (b_2 - t_{23} x_3) / t_{22}$$

$$= \frac{b_3}{\hat{t}_{33}} \quad \text{where}$$

$$x_1 = (b_1 - t_{12} x_2 - t_{13} x_3) / t_{11}$$

$$\hat{t}_{33} = \frac{t_{33}}{1 + \delta_1}$$

$$\tilde{x}_2 = (b_2 \ominus t_{23} \odot \tilde{x}_3) \oslash t_{22}$$

$$\hat{t}_{23} := t_{23} (1 + \delta_2)$$

$$\approx t_{33} (1 - \delta_1)$$

$$= (b_2 \ominus \hat{t}_{23} \tilde{x}_3 (1 + \delta_2)) \oslash t_{22} = (b_2 \ominus \hat{t}_{23} \tilde{x}_3) \oslash t_{22}$$

$$= (b_2 - \hat{t}_{23} \tilde{x}_3) (1 + \delta_3) \oslash t_{22}$$

$$= \frac{b_2 - \hat{t}_{23} \tilde{x}_3}{t_{22}} (1 + \delta_3) (1 + \delta_4) = \frac{b_2 - \hat{t}_{23} \tilde{x}_3}{\hat{t}_{22}}$$

$$\hat{t}_{22} := t_{22} \cdot \frac{1}{(1 + \delta_3)(1 + \delta_4)}$$

$$\tilde{x}_1 = (b_1 \ominus t_{12} \odot \tilde{x}_2 \ominus t_{13} \odot \tilde{x}_3) \oslash t_{11}$$

...

(a)

$\Rightarrow$ computing the solution of a least-squares problem with QR factorization is backward stable!

THEOREM: Computing the solution of least-squares problems with an SVD is also backward stable.

NORMAL EQUATIONS FOR LEAST SQUARES:

$$\min \|Ax - b\| \iff A^T A x = A^T b$$

NOT BACKWARD STABLE

- compute $c = A^T b$

- compute $B = A^T A$

- solve the linear system $Bx = c$

↳ instead of c , we will have a computed version $\tilde{c}$.

$B^{-1} \tilde{c} = \tilde{x}$ differs from x by at most $K(B) \frac{\|\tilde{c} - c\|}{\|c\|}$

and $\kappa(B) = \kappa(A)^2$

SUMMARY OF ALGORITHMS FOR LEAST-SQUARES

$$\min \|Ax - b\|_2 \quad A \in \mathbb{R}^{m \times n} \quad m \geq n$$

Normal equations

QR

SVD

cost ($m \approx n$)

$$mn^2$$

$$\frac{4}{3}n^3$$

$$13n^3$$

cost ($m \gg n$)

$$mn^2$$

$$2mn^2$$

$$2mn^2$$

stability

can be poor ;

backward stable ;

backward stable ;
+ gives you information
on the distance to
singularity

○ — ○ — ○ — ○ — ○

(1) $\kappa(A^T A) = \kappa(A)^2$ (everything is square for simplicity) $\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$

(2) $\kappa\left(\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}\right) = \kappa(A)$

B Singular values of B are the square roots of the eigenvalues of $B^T B$

(1) $A = U \Sigma V^T \rightarrow A^T A = V \underbrace{\Sigma^T \Sigma}_I V^T = V \Sigma^2 V^T$

(2) $\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} = \begin{bmatrix} AA^T & 0 \\ 0 & A^T A \end{bmatrix} =$
 $= \begin{bmatrix} U \Sigma^2 U^T & 0 \\ 0 & V \Sigma^2 V^T \end{bmatrix} = \begin{bmatrix} U & 0 \\ 0 & V \end{bmatrix} \begin{bmatrix} \Sigma^2 & 0 \\ 0 & \Sigma^2 \end{bmatrix} \begin{bmatrix} U^T & 0 \\ 0 & V^T \end{bmatrix}$

$\Rightarrow$ the sing. values of $\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}^2$ are

$$\sigma_1(A)^2, \sigma_1(A)^2, \sigma_2(A)^2, \sigma_2(A)^2, \dots$$

$\Rightarrow$ the sing. values of $\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$ are $\sigma_1(A), \sigma_1(A), \sigma_2(A), \sigma_2(A), \dots, \sigma_n(A), \sigma_n(A)$

$$\Rightarrow \kappa\left(\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}\right) = \kappa(A)$$

$\hookrightarrow$ the eigenvalues of $A^T A$,
which are also the singular values
of $A^T A$ because it's SPSP,
are $\sigma_1^2(A), \dots, \sigma_n^2(A)$

$$\Rightarrow \kappa(A^T A) = \frac{\sigma_1^2(A)}{\sigma_n^2(A)}$$